Algorithm 6 Double-Stream Chunked Streaming

1: Initialize two CUDA streams: `compute` and `memcpy`
2: $begin \leftarrow 0$, $iter \leftarrow 0$
3: Allocate `vpLB`, `vpUB` buffers of size `CHUNK_SIZE`
4: Allocate two output buffers `vBuf`[0], `vBuf`[1]
5: **while** $begin < |\texttt{oPairs}|$ **do**
6: $end \leftarrow begin$, $voxCnt \leftarrow 0$
7: $curr \leftarrow iter \bmod 2$, $prev \leftarrow 1 - curr$
8: **while** $end < |\texttt{oPairs}|$ **and** $voxCnt + \texttt{op2vpCounts}[end] \leq \texttt{CHUNK_SIZE}$ **do**
9: $voxCnt \leftarrow voxCnt + \texttt{op2vpCounts}[end]$
10: $end \leftarrow end + 1$
11: **end while**
12: **if** $iter > 0$ **then**
13: (Async) D2H copy of `vBuf`$[prev]$ via `memcpy` stream
14: **end if**
15: (Async) launch Algorithms 1 and 2 on `compute` stream, store results in `vBuf`$[curr]$
16: Enforce cross-stream buffer dependencies using CUDA events
17: $begin \leftarrow end$, $iter \leftarrow iter + 1$
18: **end while**
19: **if** $iter > 0$ **then**
20: (Async) D2H copy of the last chunk from `vBuf`$[(iter - 1) \bmod 2]$ via `memcpy` stream
21: Synchronize `memcpy` stream
22: **end if**

APPENDIX

A. Double-Stream Pipelining

Although chunked streaming bounds GPU memory usage, synchronous device-to-host (D2H) transfers after each chunk can leave the GPU idle. To overlap computation with data transfer, we employ two CUDA streams: a `compute` stream for GPU kernel execution and a `memcpy` stream for asynchronous D2H transfers. Algorithm 6 details the pipelined execution.

We allocate a single pair of GPU buffers, `vpLB` and `vpUB`, to store intermediate voxel-pair distance bounds (Line 3). These buffers can be reused across chunks because the `compute` stream processes chunks sequentially. Each chunk consists of consecutive object pairs in `oPairs`, identified by indices `begin` and `end`. We accumulate object pairs until their total voxel-pair count reaches the capacity `CHUNK_SIZE` (Lines 6 & 8–10), ensuring that the intermediate distance bounds fit within GPU memory.

Unlike the intermediate distance bounds, the surviving voxel pairs must be transferred to the host for subsequent refinement. To overlap this transfer with GPU computation, we allocate two output buffers, `vBuf`[0] and `vBuf`[1] (Line 4), and alternate their roles across iterations. At iteration `iter`, `curr` = `iter` mod 2 selects the current buffer, while `prev` = 1 − `curr` identifies the previous one (Line 7). Specifically, the `compute` stream executes Algorithms 1 and 2, writing the surviving voxel pairs into `vBuf`[`curr`] (Line 15), while the `memcpy` stream asynchronously transfers the previous chunk's results from `vBuf`[`prev`] to host memory (Lines 12–14). This allows computation and D2H transfer to overlap on different buffers, improving GPU utilization.

We use two CUDA events to ensure correct buffer access across streams. After the `compute` stream finishes writing an output buffer, it records a `computeDone` event, which the `memcpy` stream waits for before transferring its contents. Conversely, after the D2H transfer completes, the `memcpy` stream records a `copyDone` event, which the `compute` stream waits for before reusing the buffer. This prevents premature reads and overwrites without global synchronization. After processing all chunks, the final output buffer is transferred to the host (Lines 19-21).

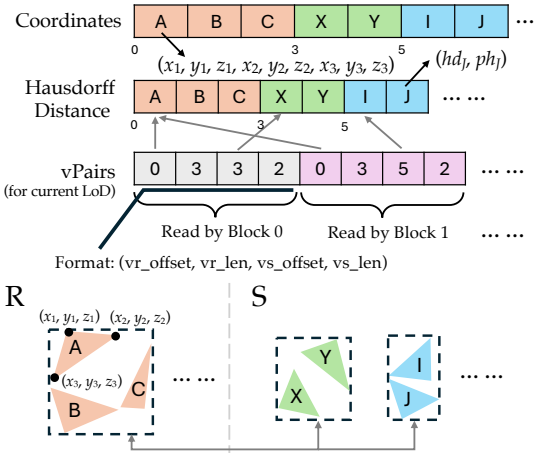


Fig. 23: Voxel and Voxel-Pair Data Layout

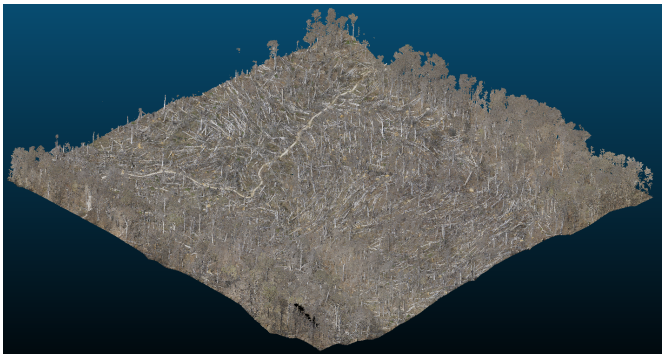
Figure 10 illustrates the overlap between computation and D2H transfers, where "C1" denotes chunk 1, and "A1" and "A2" correspond to Algorithms 1 and 2, respectively.

B. Facet-Level Voxel-Pair Data Preparation

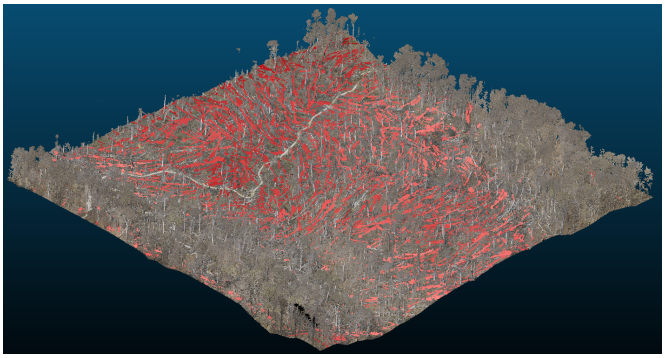
Algorithm 3 requires accessing several pieces of facet-level data for each valid voxel pair. Specifically, in Line 1, each thread block retrieves a voxel pair (v_r, v_s) from `vPairs`, which encodes the offsets and lengths of the corresponding facet arrays. Lines 2–3 obtain the numbers of triangles in the two voxels, determining the total number of facet pairs to be processed. Inside the main loop from Line 5, each thread reads the triangle coordinates of its assigned facet pair (Line 7) for distance computation. In addition, Lines 9–10 access the precomputed facet-level Hausdorff and proxy Hausdorff values (`HD` and `PH`) associated with each facet to derive tight lower and upper bounds.

To support efficient GPU execution, these data for the current LoD are first prepared on the CPU and organized into contiguous arrays following the layout illustrated in Figure 23, and then transferred to GPU global memory. Specifically, the CPU first collects the surviving voxel pairs at the current LoD, **deduplicates** their voxels to get a list of **unique** voxels, and obtains their facets which are then organized into two compact facet arrays, one for facet coordinates and the other for facet-level Hausdorff and proxy Hausdorff distances. Each voxel pair (v_r, v_s) is then represented by a tuple $(v_r^{\text{off}}, v_r^{\text{len}}, v_s^{\text{off}}, v_s^{\text{len}})$, which records the offsets and lengths of their corresponding facet segments in the two facet arrays.

This organization allows each thread block of Algorithm 3 to directly access the required data via simple pointer arithmetic.



(a) The Point Cloud of Studied Region After Tornado



(b) Downed Trees in the Tornado Corridor Highlighted in Red

Fig. 24: Tornado Corridor Visualization Using 3DPipe

C. A Real-World Case Study

We demonstrate the practical utility of 3DPipe by delineating the path of an EF-3 tornado and identifying the severely affected forest regions from large-scale 3D LiDAR data. Our study uses an aerial LiDAR survey collected after the tornado struck McCormick's Creek State Park in Spencer, Indiana, on March 31, 2023. The tornado caused widespread forest damage, leaving numerous downed trees. Figure 24a shows our studied area, covering 1398.48 m $\times$ 984.38 m.

We construct two object sets for spatial join: the standing-tree set S and the downed-tree set R. Standing trees are detected directly from the point cloud using ForestFormer3D [54]. Downed trees are extracted using a two-stage pipeline. We first apply cloth simulation filtering (CSF) [55] to remove the tree canopy and generate a bird's-eye view, where Mask2Former [56], fine-tuned using manually annotated region proposals, generates candidate downed-tree regions. The regions are back-projected to the point cloud and refined by a PointNet++ [57] model, fine-tuned using manually annotated 3D tree instances created with CloudCompare, to obtain the final 3D downed-tree instances. This process produces 3,193 standing trees and 2,451 downed trees in our studied area.

Since 3DPipe operates on watertight meshes, each segmented tree point cloud is converted into a watertight mesh using the alpha shape algorithm [58]. For each tree, we increase the alpha value from 0.5 to 6.0 and use the first value that yields a watertight mesh.

We perform a within-τ spatial join between R and S with $\tau = 60$ m. 3DPipe completes the query in 4.5 seconds. For visualization, we aggregate the join results by each downed tree to count its nearby standing trees. Downed trees with fewer than 134 neighboring standing trees are excluded from the visualization, while the remaining trees are displayed using a red color map, where darker colors indicate fewer nearby standing trees and thus locations closer to the tornado center. The distance threshold (60 m) and count threshold (134) are empirically selected to clearly delineate the tornado corridor. As Figure 24b shows, the resulting visualization successfully identifies the tornado corridor and highlights regions with concentrated tree damage, providing useful information for post-disaster assessment and management.

The efficiency of 3DPipe becomes even more important for larger-scale post-disaster assessment over entire parks or forests. For example, if R consists of hiking trails and S of downed trees, 3DPipe can accurately identify trees blocking trails to guide trail clearing.